NAME	SHARMILA S
REGISTER NO	1912260141
DEPARTMENT	B.TECH-ECE
COURSE NAME	PYTHON PROGRAMMING
COURSE CODE	CSA0823
SLOT	C
GITHUB LINK	https://github.com/ssharmila8125-netizen/sharmisCSA08-1912260141

TASK 1 – File-Processing Design Review

1.1 Diagnose the Script

1. Hard-coded file path: The program works only on the office computer. It may fail on another system.
2. File is not closed: Resources may remain occupied, causing file-handling problems.
3. Reads entire file: f.read() uses large memory for 50,000 lines.
4. Bare except: Hides all errors, making debugging difficult.
5. Incorrect amount conversion: int() cannot process amounts such as 1250.50.
6. Repeated string concatenation: Using + repeatedly is inefficient for large reports.
7. Empty lines: The last empty line may cause an indexing error.
8. No rejected-line record: Invalid records are silently ignored, leading to incorrect billing.

1.2 Repair the Design

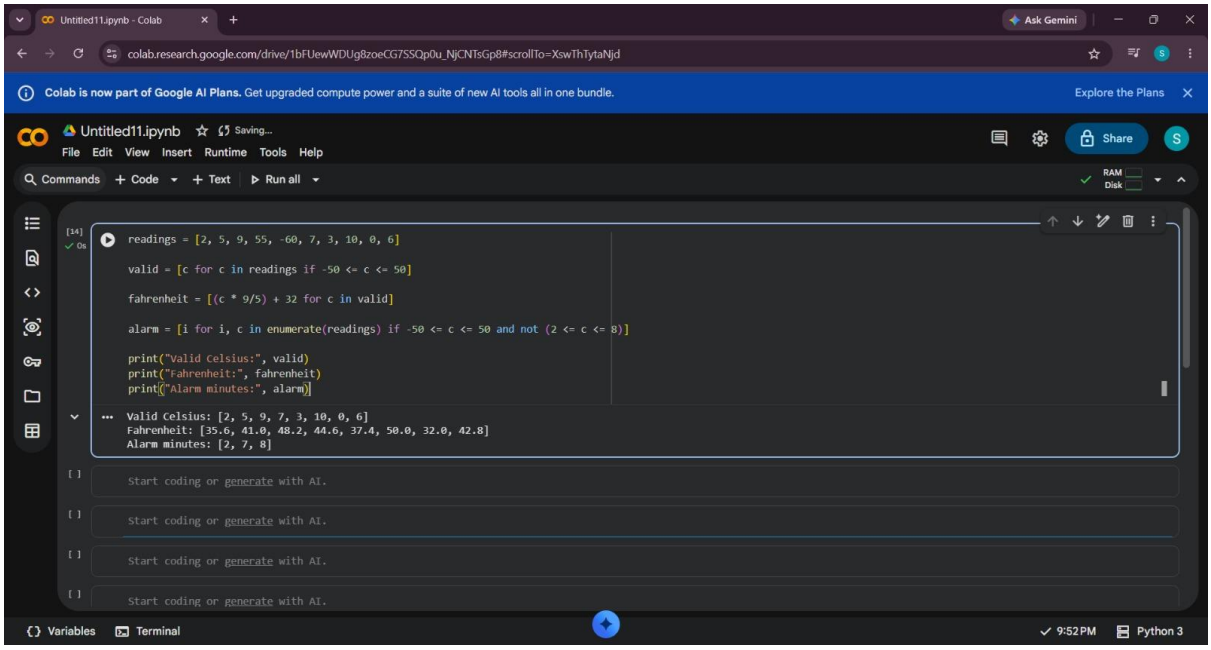
1.3 Design Defence

The repaired program is safer for 50,000 lines because it reads one line at a time, reducing memory usage. The with statement automatically closes the file. Specific exceptions identify invalid records without hiding unexpected errors. Rejected lines are stored for checking, and the function accepts a filename, making the program reusable.

Alternative Concept and Complexity Analysis

Method	Time complexity	Space complexity
read() / readlines()	O(n)	O(n)
Line-by-line reading	O(n)	O(1) extra memory per line

Here, n represents the total file size. Line-by-line processing is more memory-efficient.



TASK 2 – From Requirement to File-Processing Design

2.1 Model the Data

Input format:

RegisterNumber,Date,Status

192311001,2026-10-08,P

192311002,2026-10-08,A

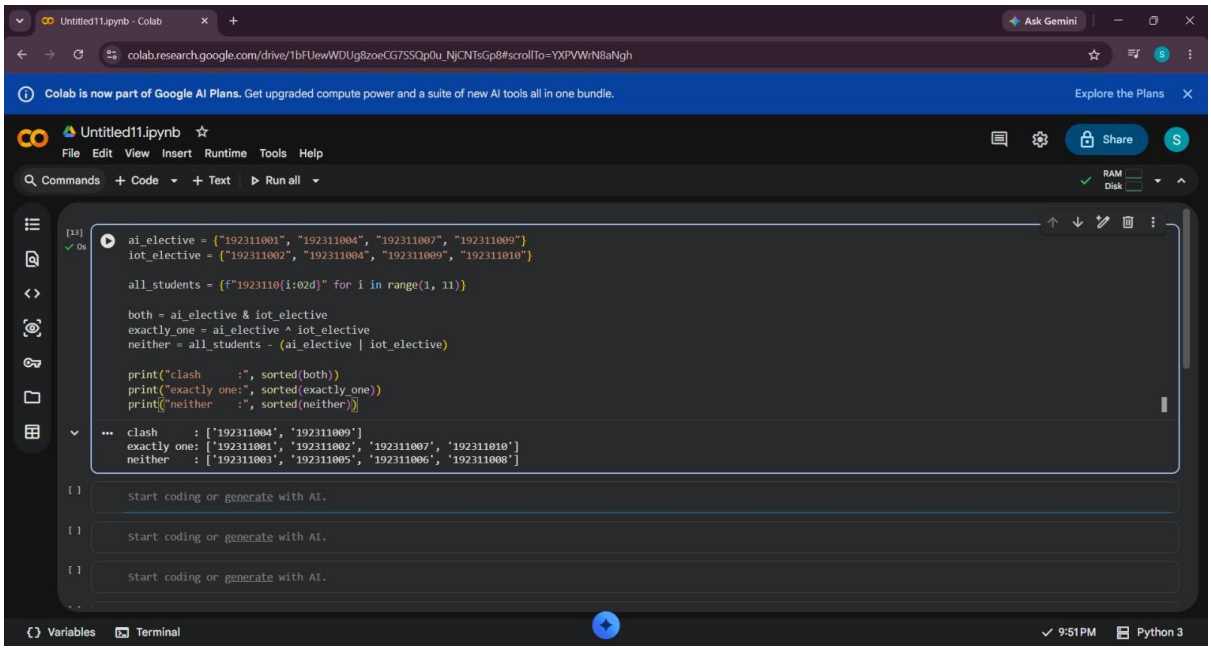
Data structures:

- Dictionary to store present and absent counts.
- List to store damaged records.
- Set to store unique student register numbers.

Output files:

1. attendance_report.csv – Register number, present days, absent days and percentage.
2. below_75.csv – Students whose attendance is below 75%.

3. errors.log – File name, line number and damaged record.



2.2 Design the File Operations

Operation	Mode / Method	Purpose
Read files	r	Read attendance records
Write reports	w	Create output reports
Add errors	a	Append errors to log
Encoding	UTF-8	Support text safely
Reading	Iteration	Process line by line
Writing	write()	Save report records

2.3 Pseudocode

BEGIN

SET attendance dictionary

SET errors list

LIST all daily files using pathlib

```
FOR each file
    OPEN file in read mode

    FOR each line with line number
        SPLIT line into register number, date, status

        IF record is valid
            IF register number not in dictionary
                INITIALISE present and absent counts

            IF status is P
                INCREMENT present count
            ELSE
                INCREMENT absent count
        ELSE
            ADD filename, line number and record to errors

    CLOSE file

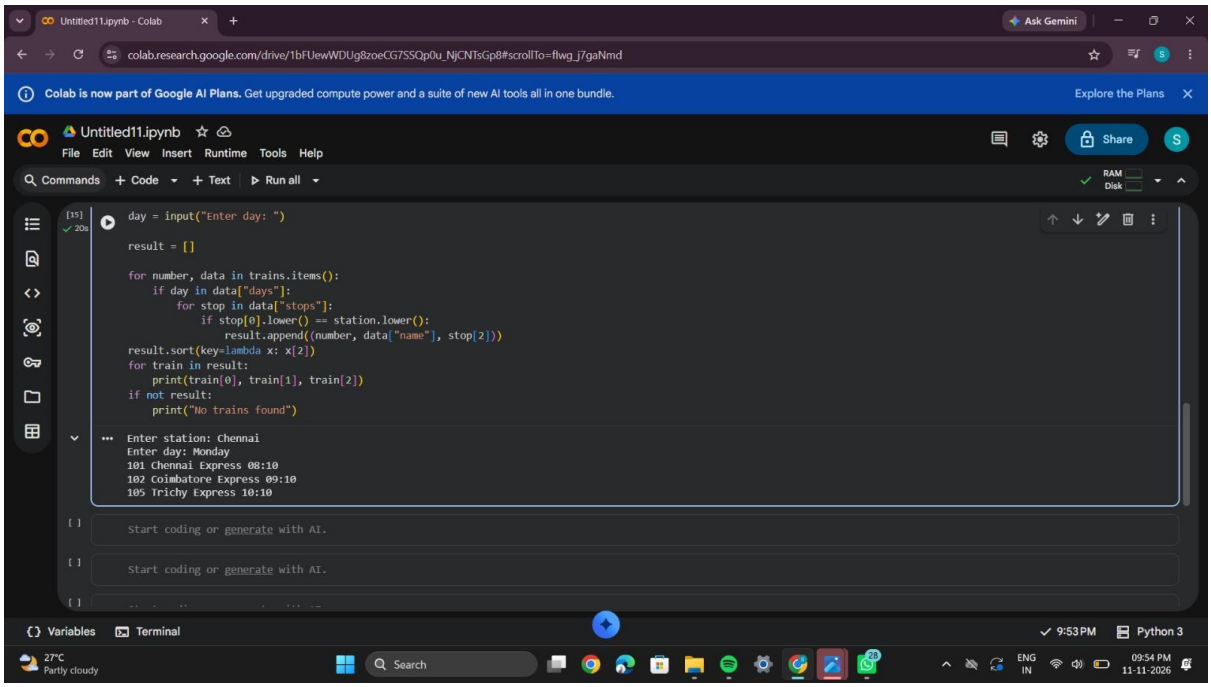
FOR each student
    CALCULATE attendance percentage

    WRITE student details to attendance report

    IF percentage is below 75
        WRITE student to below-75 report

WRITE errors to error log
```

END



2.4 Trace

Input:

192311001,2026-10-08,P

192311002,2026-10-08,A

192311003,2026-10-08

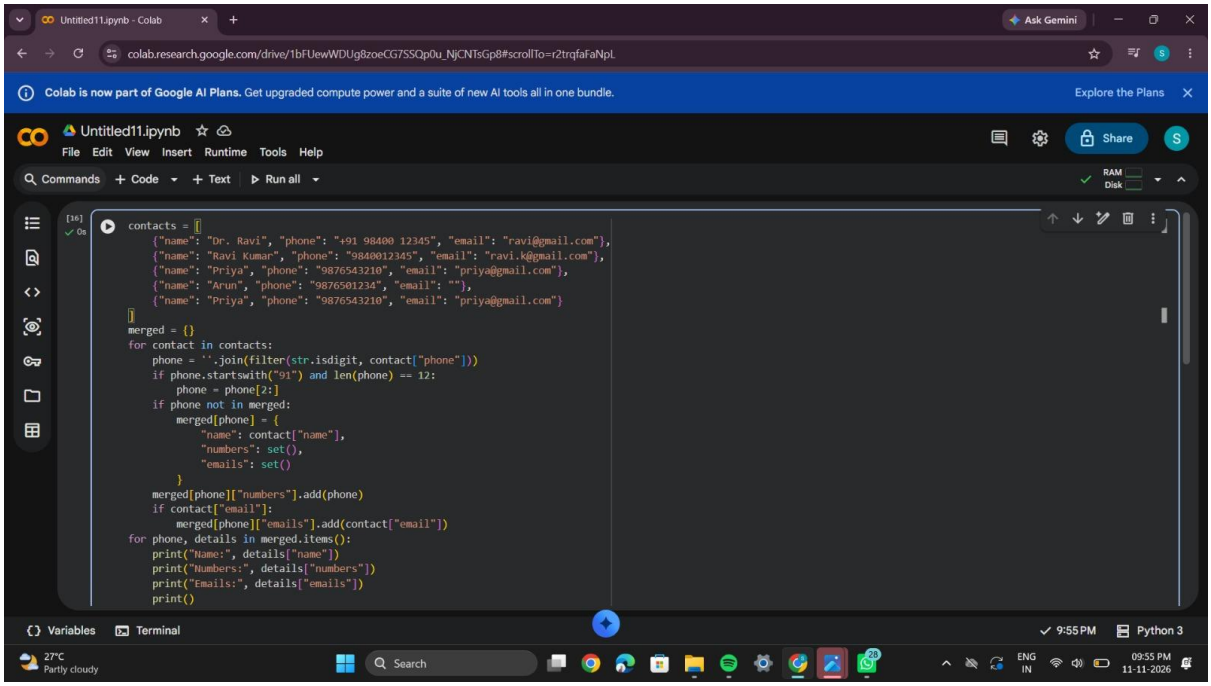
Register number	Result
192311001	Present count = 1
192311002	Absent count = 1
192311003	Invalid record logged

Alternative Concept and Complexity Analysis

Re-reading all files for every student requires O(s × f × l) time.

Reading each file once and updating a dictionary requires O(f × l) time.

Dictionary processing is faster and uses O(s) extra space for student counts.



TASK 3 – Design the Error Handling

3.1 Identify the Failures

Failure	Exception
Missing folder or file	FileNotFoundError
Permission denied	PermissionError
Wrong encoding	UnicodeDecodeError
Invalid status	ValueError
Missing fields	IndexError
Empty file	No exception; zero records
Invalid record format	Custom InvalidRecord

3.2 Design a Custom Exception

It is raised inside parse_line() when the record format is incorrect and handled inside summarise().

3.3 Complete the Code

3.4 Misuse Experiment

When except Exception: pass is used, a typing error such as an incorrect variable name is silently ignored.

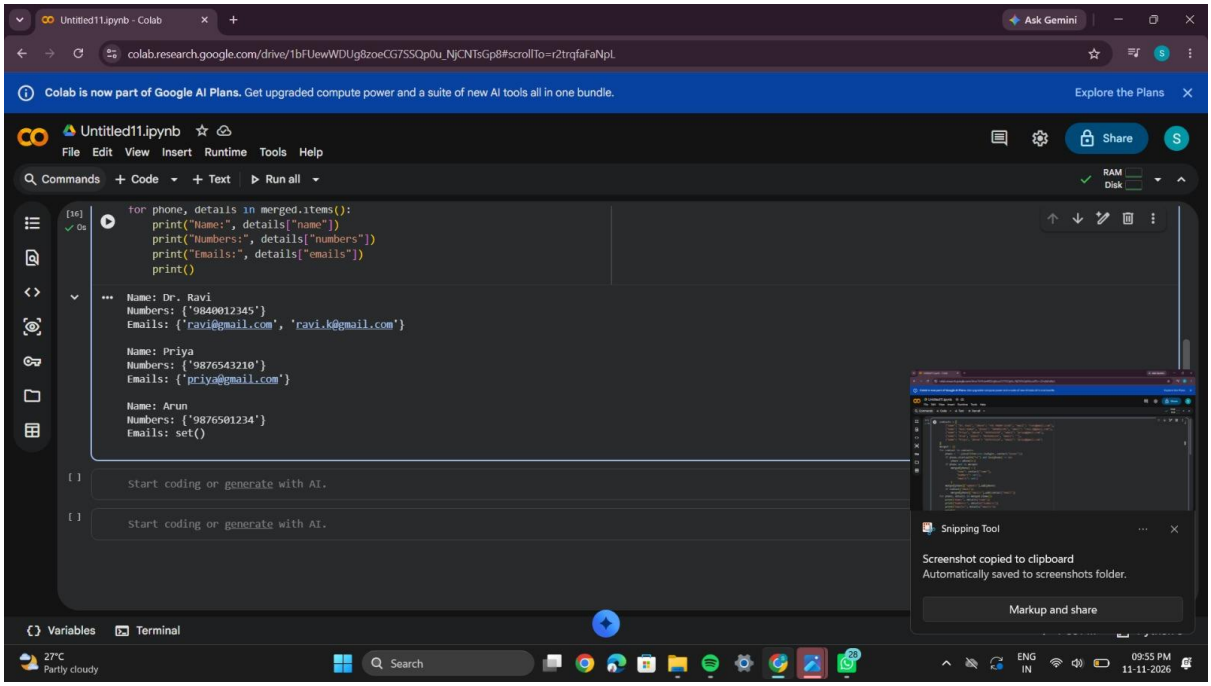
This hides unexpected programming errors and makes debugging difficult.

Alternative Concept and Complexity Analysis

- Exception-based approach: Uses try and except. When invalid records are rare, it is convenient.
- If-based approach: Checks fields and status before processing.

Both have O(n) time complexity overall.

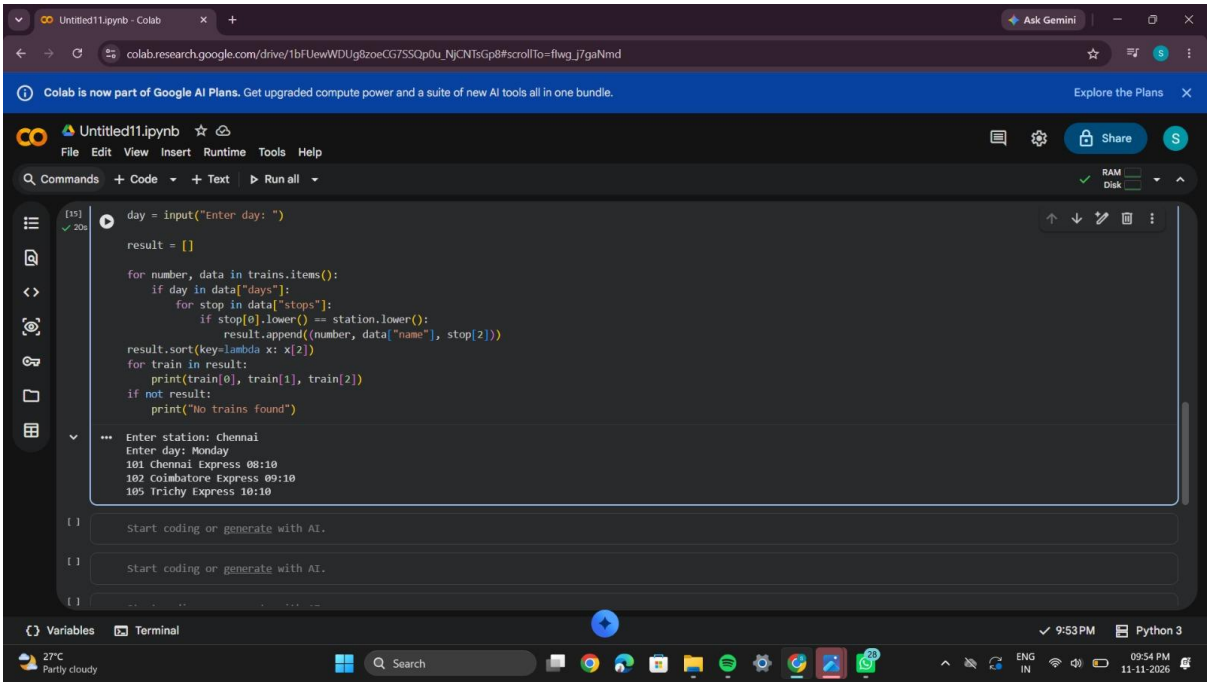
When errors are common, explicit if checks may be more efficient because exceptions have additional overhead.



TASK 4 – File and Exception Forensics

4.1 Locate the Defects

1. Earlier days are lost: Opening the file using "w" overwrites existing contents.
2. Books are joined together: write() does not automatically add newline characters.
3. File remains open if write fails: Without with or finally, the file may not close properly.

4. 

4.2 Construct Evidence

Test using three different day files:

The log contains only the third day's books because "w" overwrites the previous contents. The entries appear joined because newline characters are not written.

4.3 Repair the Program

4.4 Prevent Recurrence

1. Use with statements to automatically close files.
2. Use "a" for adding records and include newline characters between records.

Alternative Concept and Complexity Analysis

Using try/finally:

Method	Time	Space	File closure
with	O(n)	O(1)	extra Automatic
try/finally	O(n)	O(1)	extra Explicit

Both methods have the same time and space complexity. Both close the file correctly when used properly.

TASK 5 – The Same Utility Organised Four Ways

5.1 Four Organisations

(a) One Script

attendance.py

Contains reading, statistics and report functions in one file.

b) One Module and Main Script

project/

attendance.py

main.py

c) Three Modules

project/

reader.py

stats.py

report.py

main.py

- reader.py – Reads files.
- stats.py – Calculates attendance.
- report.py – Writes reports.

(d) Package Organisation

attendance/

__init__.py

reader.py

stats.py

report.py

main.py

5.2 Change the Requirement

The department needs a report for students absent for three consecutive days.

Organisation	Files changed
One script	1

One module + main	1–2
Three modules	2
Package	2

In a package, the new feature can be added mainly to stats.py and connected to report.py.

5.3 Compare the Organisations

Organisation	Readability	Reuse	Testing
One script	Low for large programs	Low	Difficult
One module	Moderate	Moderate	Moderate
Three modules	High	High	Easy
Package	Very high	Very high	Easy

Import differences:

- import attendance.stats – Imports the module; use attendance.stats.percentage().
- from attendance import stats – Imports the stats module; use stats.percentage().
- from attendance.stats import percentage – Imports the function directly; use percentage().

Alternative Concept and Complexity Analysis

A single script is simpler for small programs but difficult to maintain as the program grows.

Packages improve readability, testing and code reuse.

- Import cost: Modules are loaded once and cached in sys.modules.
- Time: Initial import requires module loading; subsequent imports reuse cached modules.
- Memory: Modules occupy memory after loading.

Finding and changing a function is easier in a package because related functions are organised separately.

TASK 6 – Design Challenge: College Fest Registration Desk

Stage 1 – Interpret the Problem

Inputs

- Five department registration files.
- Each line contains register number, name, department, email and events.
- Events are separated by semicolons.

Outputs

1. master.csv – Clean registration list.
2. Event-wise participant files.
3. summary.txt – Registration summary.
4. errors.log – Damaged records and missing files.

Testable Requirements

1. Duplicate register numbers must be rejected.
2. Invalid emails must be logged.
3. Damaged records must not stop processing.
4. Missing department files must be logged.
5. Valid records must appear in the master list.
6. Participants must be grouped by event.

Stage 2 – Processing Flow

Start

Read department files

Validate fields and email

Check duplicate register number

Invalid / Duplicate → Error log

Valid → Master list

Create event-wise files and summary

End

Stage 3 – Critical Design Decisions

Duplicate Detection

Use a set to store register numbers already processed

Exception Handling

- FileNotFoundError – Missing department file.
- PermissionError – File access denied.
- ValueError – Invalid record.
- IndexError – Missing fields.

Invalid records are logged, and processing continues.

Stage 4 – Design the Package

Package Structure

fest_registration/

__init__.py

reader.py

validator.py

processor.py

report.py

main.py

Module Functions

Module	Functions
reader.py	read_file()
validator.py	validate_record(), valid_email()
processor.py	process_records()
report.py	write_master(), write_events(), write_summary()

Pseudocode

BEGIN

CREATE empty set of register numbers

CREATE master list

CREATE event dictionary

CREATE error list

LIST all department files

FOR each department file

  TRY to open file

  FOR each line

    SPLIT line into fields

    IF fields are invalid

      LOG error

      CONTINUE

    IF register number already exists

      LOG duplicate

      CONTINUE

    IF email is invalid

      LOG invalid email

      CONTINUE

    ADD register number to set

ADD registration to master list

FOR each event

ADD student to event dictionary

END FOR

HANDLE missing file

END FOR

WRITE master list

WRITE event-wise files

WRITE summary

WRITE error log

END

Stage 5 – Implement and Demonstrate

Python Program

Demonstration Test Cases

Method	**Time complexity**	**Space complexity**
List search	**$O(n^2)$**	**O(n)**
Set search	**O(n) average**	**O(n)**

A set provides faster duplicate detection because membership checking takes O(1) average time.

Test case	**Expected result**

Valid registration	Added to master list
Duplicate register number	Logged as duplicate
Invalid email	Logged as invalid
Damaged line	Logged; processing continues
Missing department file	Logged; remaining files processed

Alternative Concept and Complexity Analysis

If duplicate registrations were checked using a list instead of a set: